

Technical Guide: Topographic Vegetation Analysis of the Grand Canyon National Park

Author: James McLoughlin

I.D Number: B01050896

Date: May 2026

Repository: <https://github.com/jjmcloughlin05/EGM722-Assessment>

1. Introduction

This package provides codes for analysing and displaying vegetation health across complex topographies – a correlation that is of great interest to conservationists and researchers. The codes combine multispectral Landsat imagery with ASTER Global Digital Elevation Models (GDEM) and use techniques including mosaicking, NDVI analysis, vectorization, and zonal statistic to quantify how vegetation health changes with elevation.

To demonstrate this codes capabilities the Grand Canyon National Park is used as a case study, which exhibits a large elevation change (>2500m) coupled with noticeable changes in vegetation density and health. This document provides instructions for setting up the virtual environment, execution steps, an overview of the analytical techniques used, and the excepted outputs.

2. Setup and Installation

2.1 System Requirements & Dependencies

The scripts in this repository were developed using Jupyter, as such a package and environment management system like Conda (Miniconda/Anaconda) is recommended for their execution. The preferred IDEs are either PyCharm or JupyterLab/Notebook.

The following dependencies are required:

- **Scripting:** Python 3.11 or later, notebook.
- **Geospatial Analysis:** geopandas, rasterio, rasterstats, shapely.
- **Data Handling:** xarray, rioxarray, numpy, pandas.
- **Analysis & Visualization:** matplotlib, scipy, matplotlib-scalebar.
- **Data Acquisition:** earthaccess.

2.2 Installation Steps

1. **Repository:** Clone the GitHub Repository, which contains all the necessary scripts and sample data to execute the codes:

<https://github.com/jjmcloughlin05/EGM722-Assessment>

2. **Environment Setup:** The .yml file in the repository includes a list of the necessary dependencies.
3. **NASA EarthAccess Authorization:** The code utilizes data from the NASA earthaccess library. To access the online library, you must have an Earthdata account. Once registered, create a .netrc file in your home directory containing your login credentials in the following format:
'machine urs.earthdata.nasa.gov login <username> password <password>'

2.3 Directory Structure

The code operates on a predefined directory structure shown in Figure 1, to facilitate file management, automatic data retrieval and any potential debugging.

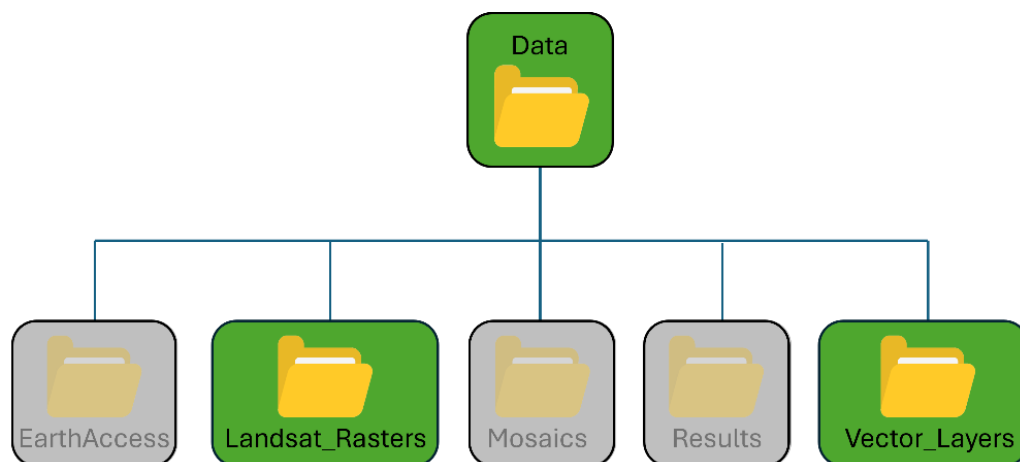


Figure1: Directory structure needed to execute the code.

Before executing the code this directory structure needs to be in place. The required setup actions to be taken depend upon how the repository was cloned:

- **Full Repository Cloned:** If the full repository was cloned then the directory structure is already configured. The only action is to provide the scripts with the file path for the parent <Data> directory (discussed later).

- **Manual Setup:** If creating the directory structure from scratch, manually create the directories indicated in **green**. Directories in **grey** are generated automatically during runtime.
 - **Note:** The name and location of the main Data directory can be changed, but all subdirectories must have the names and locations shown in figure 1 exactly. Once constructed the path of parent <Data> directory needs to be entered into the scripts (discussed later).

2.3 Sample Data

The workflow utilizes Landsat imagery, vector files, and digital elevation models (DEMs). The repository includes all necessary sample files to execute the code, but if they need to be reacquired details for their acquisition are provided below:

- **Landsat Images:** The codes processes Landsat Level-2 Surface Reflectance data only. They also do not include cloud-masking routines so images with 0% cloud cover are highly recommended.
 - **Data:** The names of the data file sets used for the study are listed below and can be obtained from the USGS EarthExplorer website (U.S. Geological Survey, 2026):
 - LC09_L2SP_037035_20250627_20250628_02_T1
 - LC09_L2SP_038035_20250704_20250705_02_T1
 - **Storage:** Place all files in <Data>/Landsat_Rasters. The code scans all subfolders automatically so manual sorting is not needed.
 - **Format:** Files must retain the NASA Landsat naming conventions. To conserve disk space only the files ending in SR_B*.TIF are needed, the rest can be discarded.
- **Vector File:** A shapefile is used to define the study area (region of interest - ROI). This is used partly to bound the region of study but it also supports memory usage by clipping rasters before processing, and provides spatial filtering when searching the earthaccess library.
 - **Data:** The study uses the Grand Canyon National Park boundary from ArcGIS online (Esri, 2024).

- **Storage:** The shapefile needs to have the following location and name:

<Data>/Vector_Layers/national_park_boundary.shp

- **Format:** The file must be in .shp format. If downloaded as an ArcGIS Pro Potral file (.pitemx) it needs to be converted to a .shp file first. This can be achieved by loading file in ArcGIS Pro or QGIS then exporting the layer.

3 Methodology

The repository contains two scripts:

- **Data_processing.ipynb:** processes the raw rasters, DEMs, and vector data, and performs the spatial analysis
- **Data_presenting.ipynb:** Visualizes the results of the analysis in the form of maps and scatter plots.

The operation and functions of these two scripts are described in the rest of this section.

3.1 Data Processing

3.1.1 Path Configuration – Required to Run Script!

Before running the script, the user must map the path of the <Data> directory in the script. This is done the ‘Base Folder Configuration’ section of the code (immediately after the module imports). To run the script as standard no further actions need to be done.

3.1.2 Determination of Required Spectral Colours

The code starts by creating a list of spectral colours required for the desired analysis and display tasks.

Note: The downloaded version of the script is set to conduct NDVI analysis and create true- and false-colour composites only. If other index analyses or composites are desired these can be added in section 2 ‘Tasks and Spectral Bands’. The titles of the tasks should be added to either the ‘DISPLAY_TASKS’ or ‘ANALYSIS_TASKS’ tuples and the task’s colours added to the ‘TASK_BANDS’ dictionary. Note that tasks names and colours should match the formatting and the order of the colours in ‘TASK_BANDS’ dictionary is important.

3.1.3 Data Cataloguing

A DataFrame of the Landsat files needed for the selected tasks is then created. This is achieved by iterating through all the Landsat data files in the '<Data>/Landsat_Rasters' directory and extracting the satellite identifier (e.g. LT07, LC09) and band identifier (e.g., B4, B5) from the filenames. These identifiers are used to determine the spectral colour of the band file (e.g., red, nir), which is checked against the list of required colours to determine whether the file is needed.

3.1.4 Mosaicking

As the study area spans multiple satellite scenes mosaicking is performed to avoid errors which can occur when analysing data across un-joined rasters. The code groups the catalogued files by colour then uses `rasterio.merge` (Rasterio, 2026a) to create a mosaic of each colour band.

3.1.5 Spatiochromatic Data Stacking

The mosaics of the colour bands are then compiled into a multi-dimensional xarray dataset (Hoyer, S. & Hamman, J., 2017). This approach is taken as it allows for the complex cross-band processes (such as NDVI analysis and composite image creation) to be achieved with relative ease compared to processing the mosaics individually. The data stacking and analysis is complimented by Dask-backed chunking, which not only employs "Lazy Loading" (i.e. objects are not loaded until they're actually needed) but enables parallel, piecewise processing that reduces the demands on memory (Dask Development Team, 2026).

3.1.6 Normalised Difference Index Calculation

The relevant spectral bands from are then selected from the multi-dimensional dataset and used to compute the Normalized Difference Vegetation Index. The code utilises the general form of the equation (equation 1) for versatility:

$$NDI = \frac{band_1 - band_2}{band_1 + band_2} \quad (1)$$

where `band_1` and `band_2` represent different spectral bands. For NDVI calculations `band_1=nir` and `band_2=red`. These specific bands are used because they utilize the unique spectral signature of chlorophyll, which strongly absorbs red light but highly reflects Near-

Infrared (NIR) light. Values range from -1 to 1, with higher values indicate denser, healthier biomass.

Note: The downloaded version of the script is set calculate NDVI only. Other index calculations (e.g. Normalized Difference Water Index: NDWI, or Normalized Difference Snow Index: NDSI) can be computed here if correctly added to 'ANALYSIS_TASKS' 'TASK_BANDS' (described in section 3.1.2).

3.1.7 DEM Acquisition and Vectorisation

The study utilises Digital Elevation Models (DEMs) to map the area's topography, a critical component for assessing NDVI as a function of height. These DEMs are retrieved from the NASA earthaccess library (NASA, 2026) using a bounding box derived from the park's boundary polygon to ensure spatial overlap (i.e. only select DEMs that overlap the study area). For the study ASTER Global Digital Elevation Models (GDEM) V3 data is used (NASA/METI et al., 2019), and files are automatically downloaded to the <Data>/EarthAccess directory then mosaicked into a single master DEM

The mosaic is categorised into 500m elevation ranges, using floor division, after which rasterio.features.shapes (Rasterio, 2026b) is used to vectorise the different 'elevation zones' into polygons.

3.1.9 Spatial Analysis

The results from the NDVI calculations and DEM vectorisation are then combined to generate the spatial data desired from this study. Here the 'zonal_stats' function within the rasterstats library (Perry, 2024) is used to calculate the mean NDVI and standard deviation within each elevation polygon. The resulting data are stored in a GeoPackage (.gpkg) in the <Data>/Results directory to allow for further analysis and visualisation.

3.1.10 Composite creation

Finally true- and false- colour composites are created from the multidimension dataset (discussed in section 3.1.5). While these are only used for display purposes it is easiest to create them in this script using the xarray dataset rather than combine them manually.

3.2 Data Presenting

The Data_presenting.ipynb script combines the composite images, DEM mosaic, and spatial statistics generated by the Data_processing.ipynb script. to display the analysis results through maps and graphs. Matplotlib (Hunter, J. D., 2007) is the package use to provide the visualisations.

3.2.1 Path Configuration and User Settings – Required to Run Script!

Similar to the Data_processing.ipynb script, the user must map the path of the <Data> directory to allow this script to locate the data needed to create the output images. In addition the user has the option of displaying either the true- or false colour composite, overlaid with either the average NDVI data or the elevation zone data.

The file mapping and display options are set in the "User Configuration" section of the script, immediately below the library imports.

3.2.2 Map Preparation and Display

After reading in the selected composite image it is immediately clipped to a predefined region to optimize memory usage. In this case the region is defined using a 10km buffer around the park shapefile's bounding box. The image contrast is then refined using percentile clipping (in this case 5% to 99%) and the brightness adjusted with gamma scaling ($\gamma = 0.8$). Finally, the clipped image is downsampled to further reduce demands on memory.

To visualise the terrain xarray-spatial (Hoyer, S. & Hamman, J., 2017) is used on the master DEM mosaic to generate a hillshade. A Gaussian filter ($\sigma = 3$) is applied to the hillshade to remove the high frequency noise and smooth the profile.

Finally the park boundary and elevation zones are added to the map alongside the chosen data layer (elevation zone values or mean NDVI).

3.2.3 Graph Preparation and Display

For the data plots, the elevation and NDVI statistics are extracted from the .gpkg file. The data is processed to calculate the mean NDVI per elevation (rather than per polygon) and plotted in a "Mean NDVI vs. Elevation" scatter plot.

4. Results

The successful execution of the analysis and visualization scripts produces the following spatial dataset and graphical outputs.

4.1 Integrated Geospatial Database (GeoPackage)

The vectorized elevation polygons and the respective NDVI statistics (mean and standard deviation) generated by the `Data_processing.ipynb` script are saved as a GeoPackage (.gpkg) to the location:

`<Data>/Results/elevation_zones_with_stats_2025.gpkg`

The .gpkg format was selected to provide a self-contained "ready-to-use" GIS layer compatible with other software packages for further mapping or analysis.

4.2 Cartographic Visualization

The `Data_presenting.ipynb` script generates a map of the area overlaid with the results of spatial analysis. An example of an output is shown in figure 2 which shows the mean NDVI values overlaid on a true-colour composite.

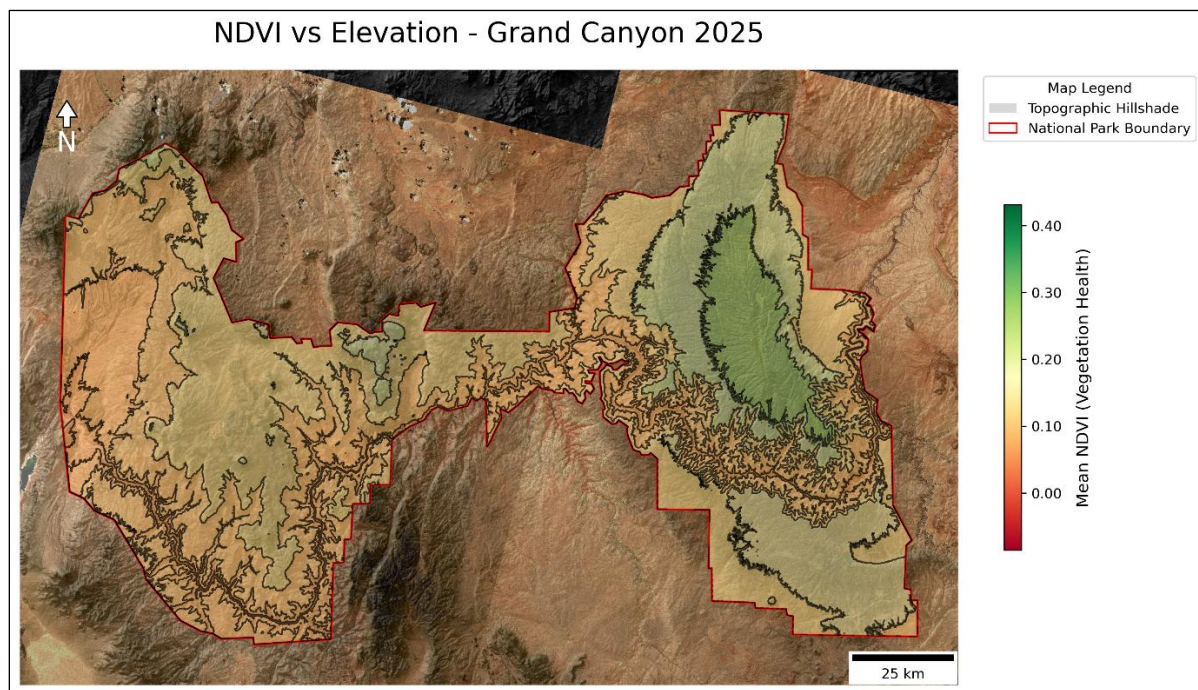


Figure 2: Example map generated by the script. The image shows a true-colour composite of the region, enhanced with a hillshade, along with the study region perimeter (red line), elevations zone boundaries (black lines), and the mean NDVI per region.

Depending upon the user's choice the output figure contains:

- **Composite Layer:** either a true-colour composite or false-colour composite.
- **Hillshade:** A smoothed topographic layer illustrating relief.
- **Segmented Overlay:** Color coded regions corresponding to either the mean vegetation health (NDVI) or elevation ranges.
- **Cartographic Elements:** A scale bar, north arrow, and legend.

The images are saved to <Data>/Results directory using the following naming convention:

<composite_type>_<data_type>_MAP_<year>.png

4.3 Statistical Elevation Profile

The Data_presenting.ipynb script also generates a scatter plot illustrating mean NDVI as a function of elevation, as illustrated in figure 3:

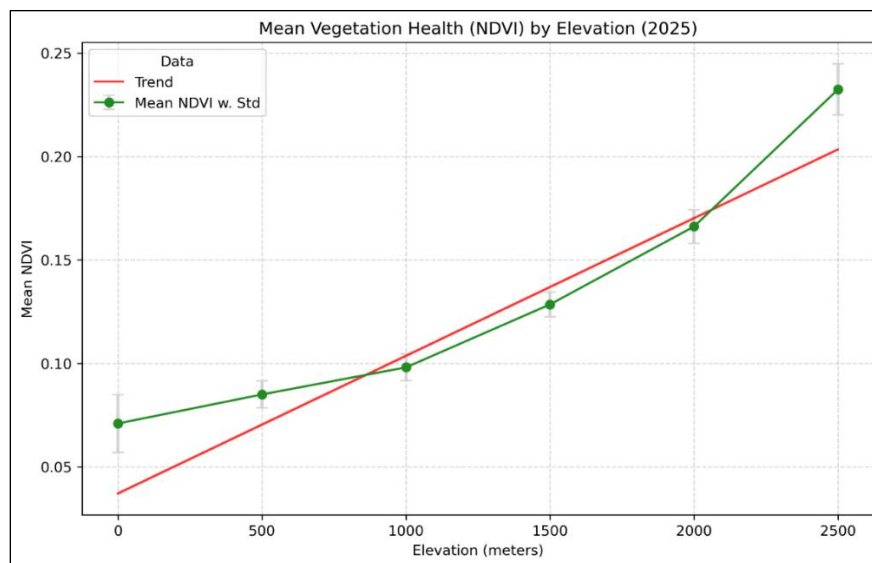


Figure 3: Scatter plot showing the mean NDVI and standard deviation per elevation step (green), along with a first-order polynomial fit (red).

The scatter plot includes:

- **Data Points with Error Bars:** Data points represent the average NDVI at each elevation step while the error bars indicate the standard deviation (spread) of the data.

- **Linear Trendline:** A first-order polynomial fit characterising the overall relationship between vegetation health and elevation within the study area.

5. Troubleshooting

Below are errors that may be encountered and steps to resolve them.

- **DataSourceError: C:\...\...: No such file or directory**
 - **Cause:** A specific file or directory is missing from your local machine, is misspelt, or is in the wrong sub-directory.
 - **Fix:** Identify the missing filename or directory from the error message. Refer to Sections 2.3 and 2.4 to verify the correct location and naming of the problem directory/file and correct accordingly.
- **KeyError: 'year'**
 - **Cause:** The script is attempting to process an empty folder or cannot find the expected Landsat files.
 - **Fix:** Ensure the <Data> path is entered correctly in the script and the Landsat_Rasters folder is named exactly as specified. Verify that the folder contains the necessary .tif files.
- **MemoryError: Unable to assign xx Mb/Gb to memory**
 - **Cause:** The raster data is too large for your system's available RAM.
 - **Fix:**
 - In the data_processing script: Reduce the chunk size in Section 5 of the code.
 - In the data_presenting script: Reduce the image resolution by increasing the x and y values in the coarsening function at the end of Section 2.
- **LoginAttemptFailure: Authentication with Earthdata Login failed**
 - **Cause:** Incorrect NASA Earthdata credentials or a malformed search geometry.
 - **Fix:** Double-check your username and password in the .netrc file.

- **Empty Map/Overlay**

- **Cause:** CRS Mismatch between the data being combined (either: rasters, ROI shapefile, DEMs, elevation polygons)
- **Fix:** Ensure the CRS match for all components being considered. Reproject any misaligned layers using `.to_crs(<img>.rio.crs)`.

6. References

- Dask Development Team (2026) Dask: Library for dynamic task scheduling (v2026.4.1) [Software]. Available at: <https://dask.org> (Accessed: 25 April 2026).
- Esri (2024) World Imagery [Online]. Available at: <https://www.arcgis.com/home/item.html?id=707bfb056d044dcac5b39f59c665647> (Accessed: 15 April 2026)
- Hoyer, S. & Hamman, J. (2017). xarray: N-D labeled arrays and datasets in Python. Journal of Open Research Software. [DOI: 10.5334/jors.148]
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering.
- NASA (2026) Earthdata Search [Online]. Available at: <https://search.earthdata.nasa.gov/> (Accessed: 4 May 2026).
- NASA/METI/AIST/Japan Spacesystems and U.S./Japan ASTER Science Team (2019) ASTER Global Digital Elevation Model V003 [Dataset]. NASA EOSDIS Land Processes DAAC. Available at: <https://doi.org/10.5067/ASTER/ASTGTM.003> (Accessed: 4 May 2026).
- Perry, M. (2024) rasterstats: Summarize geospatial raster datasets based on vector geometries (v0.19.0) [Software]. Available at: <https://github.com/perrygeo/python-rasterstats> (Accessed: 22 April 2026).
- Rasterio (2026a) rasterio.merge module. Available at: <https://rasterio.readthedocs.io/en/stable/api/rasterio.merge.html> (Accessed: 22 April 2026).
- Rasterio (2026b) rasterio.features module. Available at: [readthedocs.io](https://rasterio.readthedocs.io/en/stable/api/rasterio.features.html) (Accessed: 24 April 2026).
- U.S. Geological Survey (2026) EarthExplorer [Online]. Available at: <https://earthexplorer.usgs.gov/> (Accessed: 8 April 2026).